

Day 39: Binary Search Tree (BST) 🌳

◆ 1. BST kya hota hai?

Binary Search Tree ek special type ka Binary Tree hota hai jisme:

📌 Rule:

Left subtree → values smaller hoti hain root se

Right subtree → values greater hoti hain root se

Ye rule har node par apply hota hai

◆ Example BST:

```
    50
   /  \
  30   70
 / \  / \
20 40 60 80
```

◆ BST ke Operations

1. Insert
2. Search
3. Delete
4. Traversal (Inorder gives sorted output)

◆ 2. BST Insertion Code (Java)

```
1. package Day_39;
2.
3. class Node {
4.     int data;
5.     Node left, right;
6.
7.     Node(int data) {
8.         this.data = data;
9.         left = right = null;
10.    }
11. }
12.
13. class BST {
14.
15.     Node root;
16.
17.     // Insert function
18.     Node insert(Node root, int key) {
19.         if (root == null) {
20.             return new Node(key);
21.         }
22.
23.         if (key < root.data) {
24.             root.left = insert(root.left, key);
25.         } else {
26.             root.right = insert(root.right, key);
27.         }
28.
29.         return root;
30.     }
31.
32.     // Inorder traversal (sorted output)
33.     void inorder(Node root) {
34.         if (root != null) {
35.             inorder(root.left);
36.             System.out.print(root.data + " ");
37.             inorder(root.right);
38.         }
39.     }
40.
41.     public static void main(String[] args) {
42.         BST tree = new BST();
43.
44.         tree.root = tree.insert(tree.root, 50);
45.         tree.root = tree.insert(tree.root, 30);
46.         tree.root = tree.insert(tree.root, 70);
47.         tree.root = tree.insert(tree.root, 20);
48.         tree.root = tree.insert(tree.root, 40);
49.         tree.root = tree.insert(tree.root, 60);
50.         tree.root = tree.insert(tree.root, 80);
51.
52.         System.out.println("Inorder (Sorted): ");
53.         tree.inorder(tree.root);
54.     }
55. }
56.
```

◆ Output:

Inorder (Sorted):

20 30 40 50 60 70 80

◆ 3. BST Search Code

```
boolean search(Node root, int key) {  
    if (root == null) return false;  
  
    if (root.data == key) return true;  
  
    if (key < root.data)  
        return search(root.left, key);  
    else  
        return search(root.right, key);  
}
```

◆ 4. BST Deletion (Concept)

Deletion ke 3 cases hote hain:

Case 1: Leaf node (no child)

→ direct delete

Case 2: One child

→ child ko replace kar do

Case 3: Two children

→ inorder successor (smallest in right subtree) replace